Agenda de Contactos con Acceso Aleatorio

Carlos Andrés Cholotío Mendoza - 1517925

Este documento explica el diseño y la implementación de una agenda de contactos de escritorio en Python que almacena sus datos en un archivo de registros de tamaño fijo. La aplicación llega directamente a cualquier contacto mediante acceso aleatorio, se apoya en un índice primario en JSON, exporta a JSON y PDF, verifica la integridad con SHA-256 y genera respaldos comprimidos en ZIP.

Archivos que maneja la aplicación:

Archivo	Función
contactos.dat	Datos: un registro de 100 bytes por contacto.
indice.json	Índice primario: relaciona cada ID con su número de registro.
contactos.sha256	Hash SHA-256 de contactos.dat para detectar alteraciones externas.
contactos_exportados.json / reporte_contactos.pdf	Exportación de todos los contactos y reporte generado a partir del JSON.
respaldo_*.zip	Copia comprimida de contactos.dat, indice.json y contactos.sha256.

2. Estructura del registro

Cada contacto ocupa exactamente 100 bytes. Los campos se escriben uno detrás de otro, sin separadores, rellenando con espacios hasta su largo máximo (método ljust). Si un valor excede el largo del campo, la aplicación lo rechaza en el formulario antes de guardarlo.

Campo	Descripción	Posición inicial	Largo (bytes)
id	Número entero único	0	5
nombre	Nombre completo	5	30
telefono	Teléfono (7-15 dígitos, + - ( ))	35	15
correo	Correo electrónico	50	30
ciudad	Ciudad	80	20
Total	**Tamaño del registro**	**—**	**100**

Con esta estructura, el registro número *n* (empezando en 0) comienza en el byte **n × 100**, y el tamaño del archivo siempre es (número de registros) × 100.

3. Acceso aleatorio con seek() y tell()

Para leer un contacto no se recorre el archivo: se calcula su posición y se salta directamente a ella con seek(). La aplicación muestra además el valor de tell() para evidenciar la posición actual dentro del archivo.

3.1 Lectura (buscar por ID)

```python
def buscar_contacto(id_c, traza=None):
    """Busca por ID: índice -> seek() directo al registro (sin leer todo el archivo)."""
    indice = cargar_indice()
    for _ in range(2):
        if id_c not in indice:
            return None
        num = indice[id_c]
        pos = num * TAMANO_REGISTRO
        with open(ARCHIVO_DATOS, "rb") as f:
            f.seek(pos)                     # salto directo
            raw = f.read(TAMANO_REGISTRO)
            fin = f.tell()                  # tell(): posición tras leer
        if len(raw) == TAMANO_REGISTRO:
            c = parsear_registro(raw)
            if c["id"] == id_c:
                if traza is not None:
                    traza.update(registro=num, seek=pos, tell=fin)
                return c
        indice = reconstruir_indice()       # índice desactualizado: se reconstruye
    return None
```

3.2 Modificación en el lugar

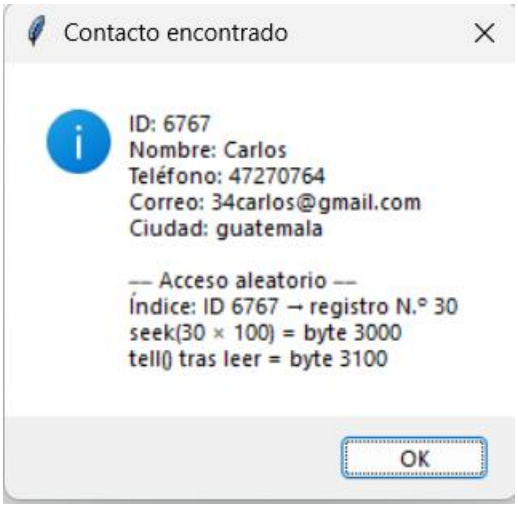
```python
def modificar_contacto(datos):
    """Sobrescribe solo el registro indicado (modificación en el lugar)."""
    indice = cargar_indice()
    if datos["id"] not in indice:
        raise ValueError("El ID no existe.")
    num = indice[datos["id"]]
    with open(ARCHIVO_DATOS, "r+b") as f:
        f.seek(num * TAMANO_REGISTRO)
        pos = f.tell()
        f.write(formatear_registro(datos))
        fin = f.tell()
    actualizar_hash()
    return {"registro": num, "seek": pos, "tell": fin}
```

3.3 Inserción (al final del archivo)

```python
def agregar_contacto(datos):
    """Añade al final del archivo. Devuelve la traza de seek()/tell()."""
    indice = cargar_indice()
    if datos["id"] in indice:
        raise ValueError("El ID ya existe.")
    if not os.path.exists(ARCHIVO_DATOS):
        open(ARCHIVO_DATOS, "wb").close()
    with open(ARCHIVO_DATOS, "r+b") as f:
        f.seek(0, os.SEEK_END)
        pos = f.tell()                      # tell(): posición donde se escribirá
        num = pos // TAMANO_REGISTRO
        f.write(formatear_registro(datos))
        fin = f.tell()                      # tell(): posición tras escribir
    indice[datos["id"]] = num
    guardar_indice(indice)
    actualizar_hash()
    return {"registro": num, "seek": pos, "tell": fin}
```

3.4 Ejemplo trazado

Esta información aparece en el cuadro «Contacto encontrado» (Contactos → Buscar por ID) y, para agregar y modificar, en la barra de estado de la ventana principal.



Búsqueda por ID mostrando registro, seek() y tell()

4. Índice primario

El índice es un archivo JSON que asocia cada ID con su número de registro dentro de contactos.dat:

```
{"1": 0, "2": 1, "3": 2, "4": 3, "5": 4}
```

Funcionamiento:

- **Agregar:** se escribe el registro al final y se añade la entrada ID → N.º de registro; el índice se guarda de forma atómica (archivo temporal + os.replace).

- **Buscar:** se consulta el índice; si el ID no existe se avisa al usuario sin tocar contactos.dat; si existe, se hace seek() al registro.
- **Recuperación:** si indice.json no existe, está corrupto o apunta a un registro que no corresponde, la aplicación lo reconstruye leyendo contactos.dat y no se cae.

¿Por qué es más eficiente que una búsqueda secuencial?

Sin índice habría que leer registros desde el inicio hasta encontrar el ID: en promedio la mitad del archivo (N/2 registros) y, en el peor caso, todo. Con el índice, la ubicación se obtiene con una búsqueda en un diccionario (tiempo constante) y se lee un solo registro de 100 bytes, sin importar cuántos contactos haya.

Contactos (N)	Secuencial (promedio)	Secuencial (peor caso)	Con índice + seek()
100	5 000 bytes	10 000 bytes	100 bytes
10 000	500 000 bytes	1 000 000 bytes	100 bytes
1 000 000	50 000 000 bytes	100 000 000 bytes	100 bytes

El costo de leer datos del archivo pasa de crecer con N a ser constante. Hay un costo menor: mantener el índice (una entrada pequeña por contacto) y actualizarlo en cada alta; en una aplicación más grande el índice se conservaría en memoria en lugar de releerlo en cada búsqueda.

5. Integridad con SHA-256

Cada vez que se guarda un cambio (agregar, modificar o eliminar) se calcula el SHA-256 de contactos.dat y se guarda en contactos.sha256. Al iniciar la aplicación, y desde Herramientas → Verificar integridad, se recalcula y se compara con el valor guardado. Si no coinciden, se avisa que el archivo fue modificado fuera de la aplicación.

```
def _hash_archivo():
    sha = hashlib.sha256()
    with open(ARCHIVO_DATOS, "rb") as f:
        for bloque in iter(lambda: f.read(65536), b""):
            sha.update(bloque)
    return sha.hexdigest()

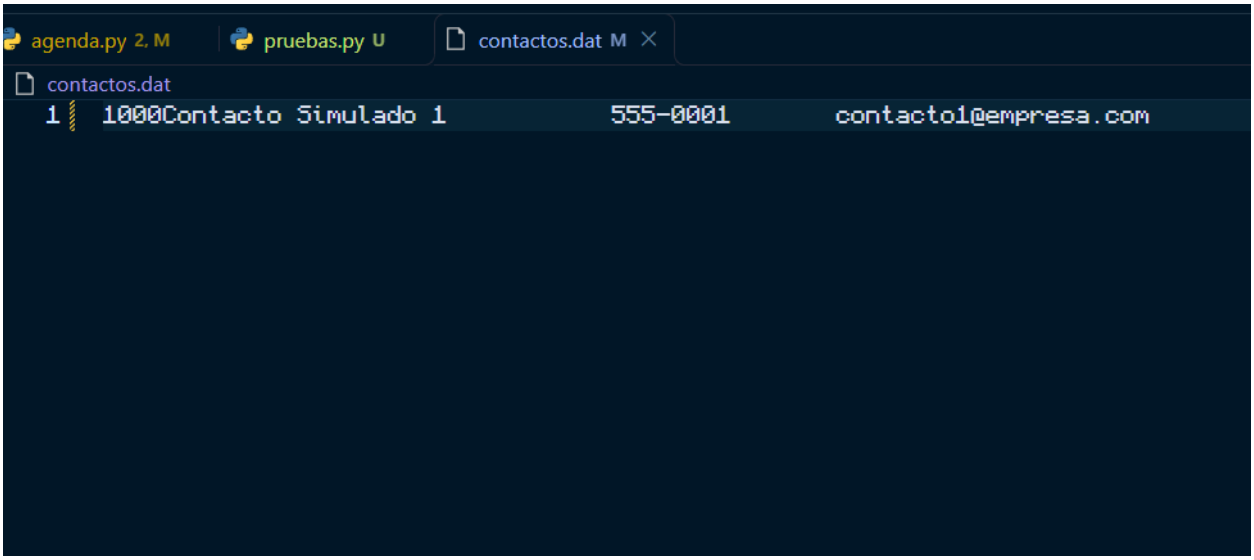

def actualizar_hash():
    if os.path.exists(ARCHIVO_DATOS):
        with open(ARCHIVO_HASH, "w", encoding="utf-8") as f:
            f.write(_hash_archivo())


def verificar_integridad():
    if not os.path.exists(ARCHIVO_DATOS) or not os.path.exists(ARCHIVO_HASH):
        return True
    with open(ARCHIVO_HASH, "r", encoding="utf-8") as f:
        return f.read().strip() == _hash_archivo()
```

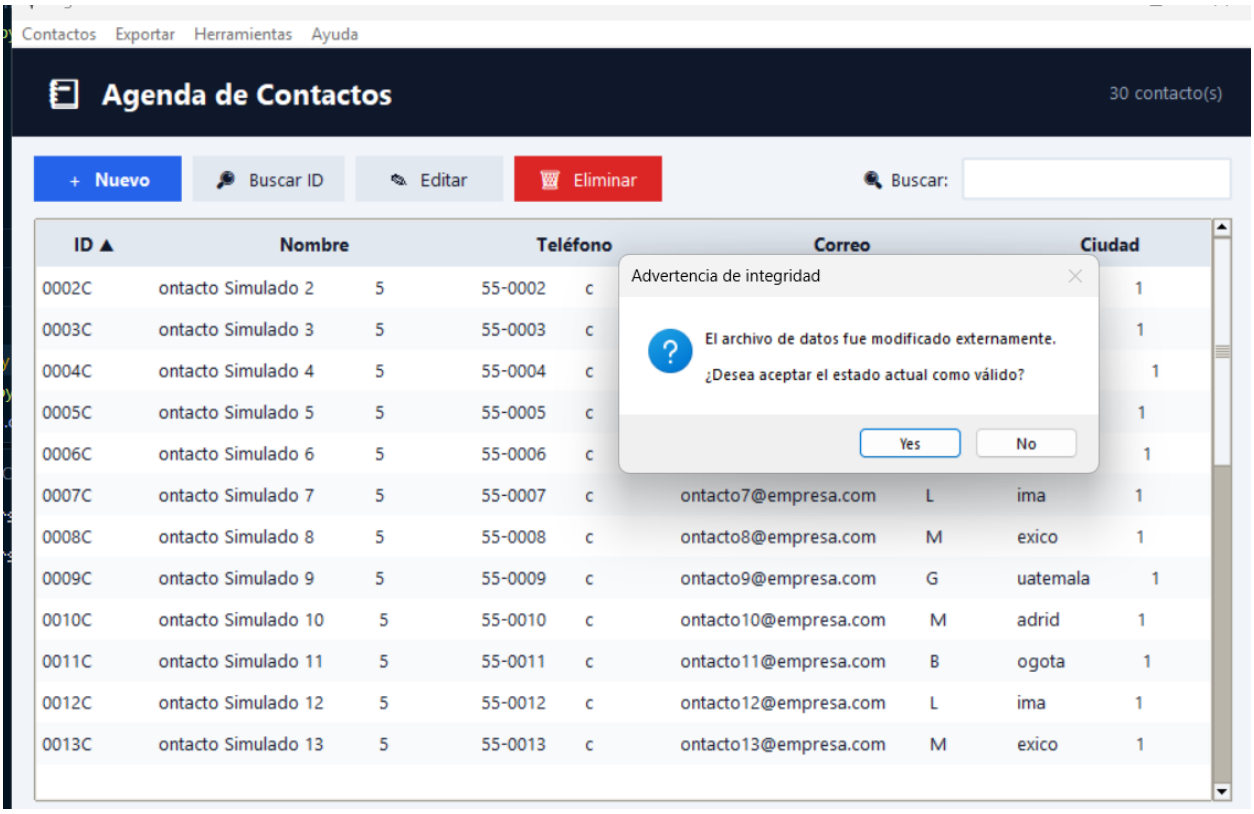
5.1 Evidencia: detección de un archivo alterado

Procedimiento usado para la prueba:

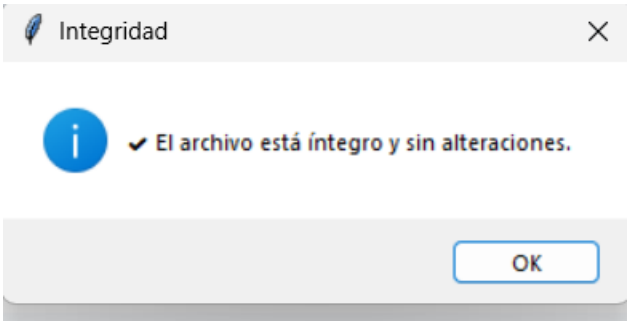
- Con la aplicación cerrada, abrir contactos.dat en un editor de texto y cambiar un carácter (por ejemplo, un dígito del teléfono). Guardar.
- Abrir la aplicación: aparece la advertencia de modificación externa.
- Ejecutar Herramientas → Verificar integridad: se informa que el archivo fue alterado.



Archivo contactos.dat modificado manualmente en el editor



Advertencia al iniciar la aplicación



Resultado de Herramientas → Verificar integridad

6. Respaldo comprimido (ZIP)

Herramientas → Crear respaldo genera un ZIP (módulo zipfile, compresión DEFLATE) con contactos.dat, indice.json y contactos.sha256, y muestra el tamaño original y el comprimido.

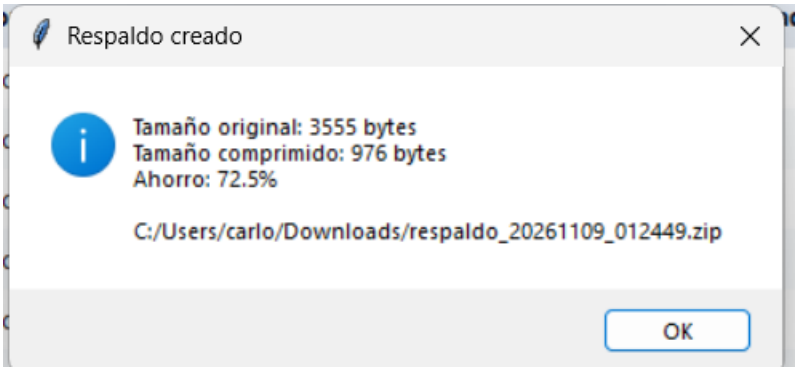
```
with zipfile.ZipFile(ruta, "w", zipfile.ZIP_DEFLATED) as z:
    for a in archivos:
        z.write(a)
original = sum(os.path.getsize(a) for a in archivos)
comprimido = os.path.getsize(ruta)
```

6.1 Comparación de tamaños

Completar con los valores reales mostrados por la aplicación. *El tamaño de contactos.dat debe ser (N.º de contactos) × 100 y el de contactos.sha256 es siempre 64 bytes.*

Archivo	Tamaño original (bytes)
contactos.dat	**[N × 100]**
indice.json	**[bytes]**
contactos.sha256	64
Total original	
respaldo (.zip)	**[bytes comprimidos]**
Ahorro	**[%]**

El ZIP resulta mucho más pequeño porque el archivo de datos contiene muchos espacios de relleno repetidos, que DEFLATE comprime de forma muy eficiente; el hash (64 caracteres) casi no se reduce.

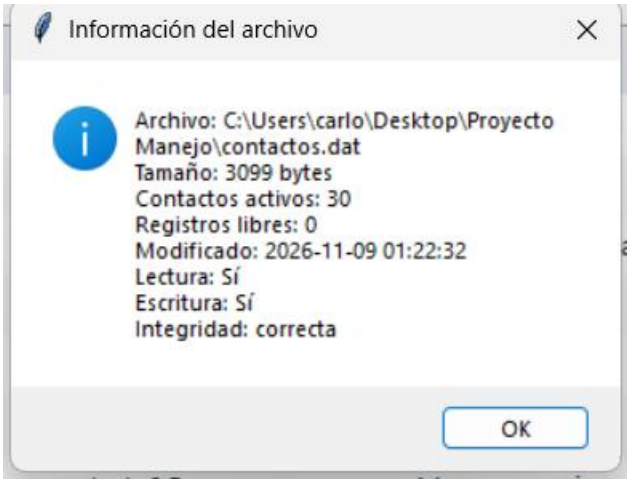


Mensaje de la aplicación con tamaño original y comprimido

7. Metadatos y manejo de errores

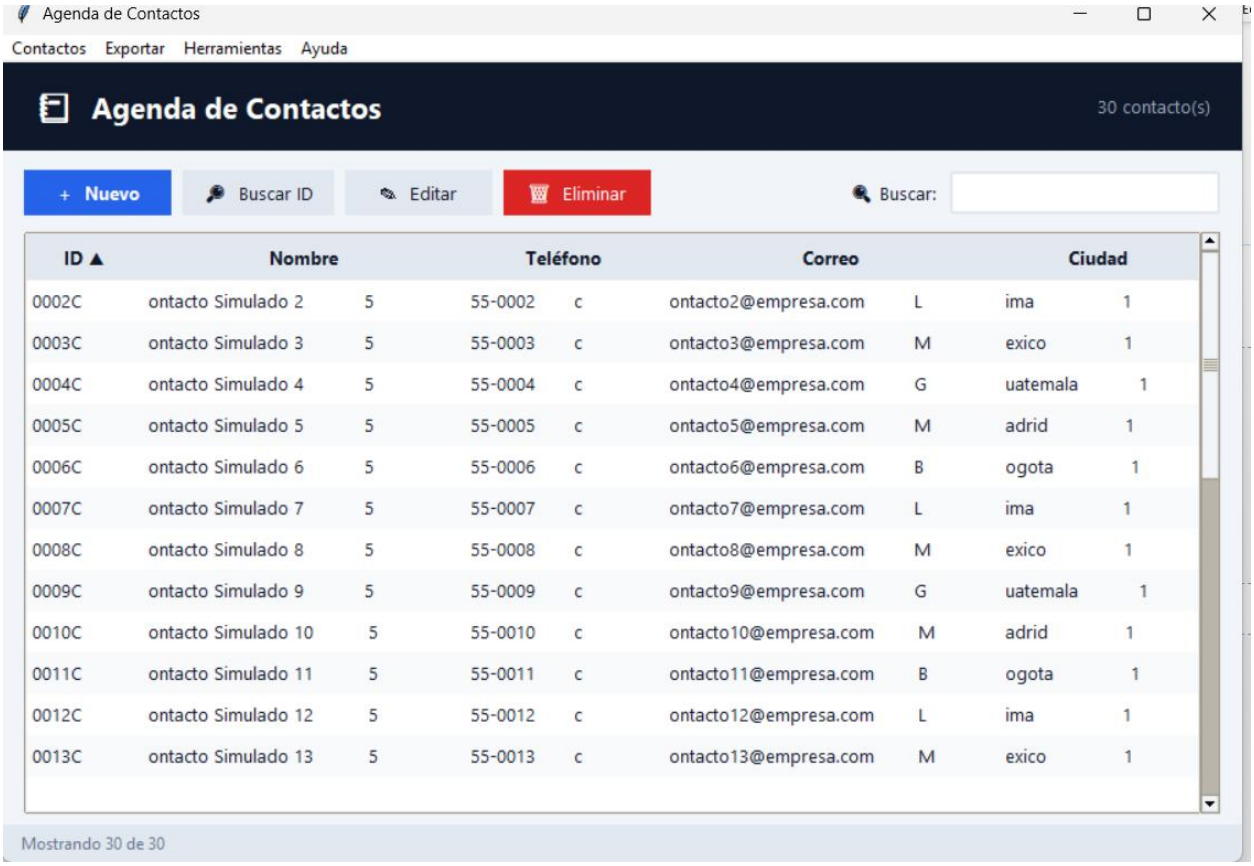
Herramientas → Información del archivo muestra el tamaño, la fecha de última modificación y los permisos de lectura y escritura, obtenidos con os.stat() y os.access(), además del número de contactos y la integridad.

Situación	Comportamiento de la aplicación
contactos.dat no existe	Se muestra la agenda vacía; el archivo se crea al agregar el primer contacto.
indice.json no existe o está corrupto	Se reconstruye automáticamente leyendo contactos.dat.
El índice apunta a un registro que no corresponde	Se detecta al comparar el ID leído, se reconstruye el índice y se reintenta.
ID inexistente al buscar	Aviso «El ID … no existe en la agenda».
ID repetido, no numérico o campos demasiado largos	El formulario muestra el error y no guarda.
Sin permisos de lectura o escritura	Cuadro de error; la aplicación no se cierra.
contactos.dat alterado externamente	Advertencia al iniciar y en Verificar integridad; se puede aceptar el estado actual.

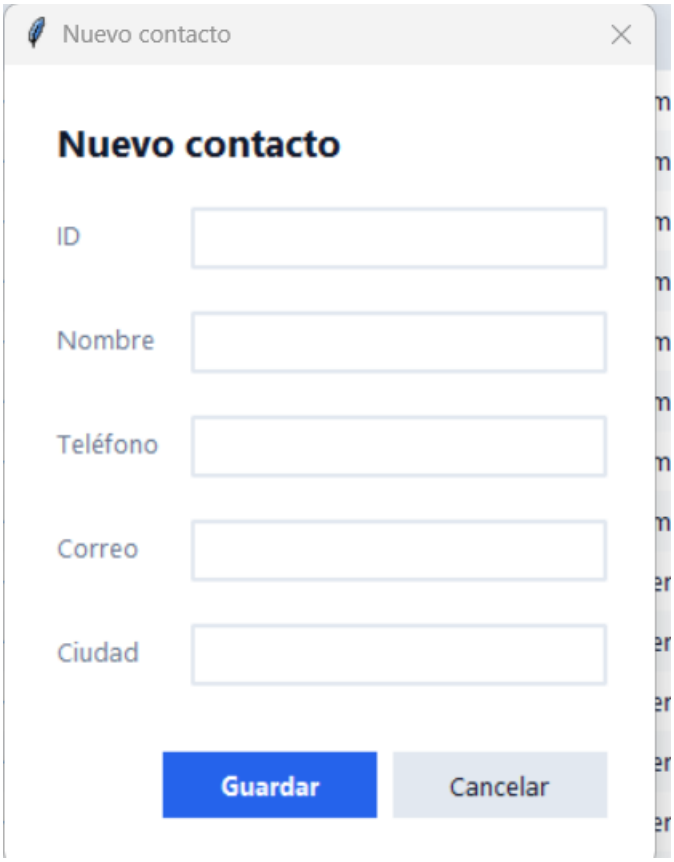


Información del archivo (metadatos y permisos)

8. Capturas del funcionamiento



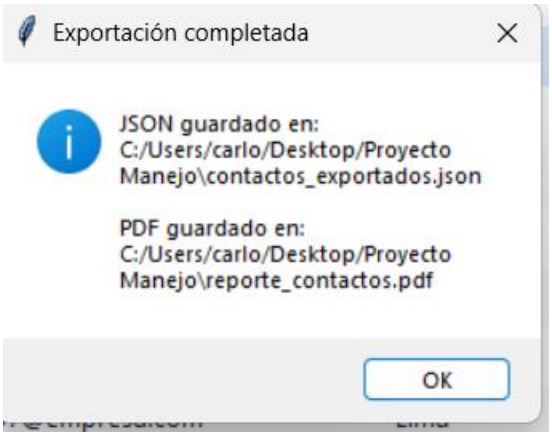
Ventana principal con la lista de contactos



Formulario «Nuevo contacto» con validación



Modificación de un contacto



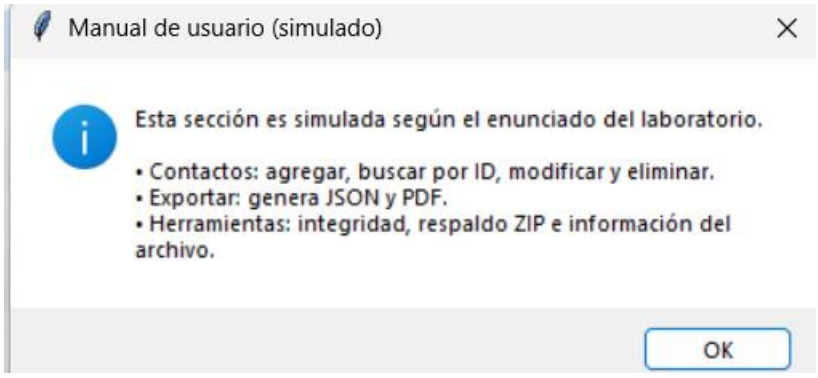
Exportación a JSON y PDF (mensaje y archivos generados)

Reporte de Contactos

2026-11-09 01:29:46

ID	Nombre	Teléfono	Correo	Ciudad
6767	Carlos	47270764	34carlos@gmail.com	guatemala
10001	Contacto Simulado 1	555-0001	contacto1@empresa.com	Bogota
10010	Contacto Simulado 10	555-0010	contacto10@empresa.com	Madrid
10011	Contacto Simulado 11	555-0011	contacto11@empresa.com	Bogota
10012	Contacto Simulado 12	555-0012	contacto12@empresa.com	Lima
10013	Contacto Simulado 13	555-0013	contacto13@empresa.com	Mexico
10014	Contacto Simulado 14	555-0014	contacto14@empresa.com	Guatemala
10015	Contacto Simulado 15	555-0015	contacto15@empresa.com	Madrid
10016	Contacto Simulado 16	555-0016	contacto16@empresa.com	Bogota
10017	Contacto Simulado 17	555-0017	contacto17@empresa.com	Lima
10018	Contacto Simulado 18	555-0018	contacto18@empresa.com	Mexico
10019	Contacto Simulado 19	555-0019	contacto19@empresa.com	Guatemala
10002	Contacto Simulado 2	555-0002	contacto2@empresa.com	Lima
10020	Contacto Simulado 20	555-0020	contacto20@empresa.com	Madrid
10021	Contacto Simulado 21	555-0021	contacto21@empresa.com	Bogota
10022	Contacto Simulado 22	555-0022	contacto22@empresa.com	Lima
10023	Contacto Simulado 23	555-0023	contacto23@empresa.com	Mexico
10024	Contacto Simulado 24	555-0024	contacto24@empresa.com	Guatemala
10025	Contacto Simulado 25	555-0025	contacto25@empresa.com	Madrid
10026	Contacto Simulado 26	555-0026	contacto26@empresa.com	Bogota

Reporte PDF generado



Menú Ayuda (simulado)

9. Conclusiones

- Los registros de tamaño fijo permiten calcular la posición de cualquier contacto (N.º de registro × 100) y acceder a él con seek() sin leer el resto del archivo.

- El índice en JSON convierte la búsqueda por ID en una consulta de tiempo constante seguida de una única lectura de 100 bytes.
- El hash SHA-256 permite detectar cualquier modificación hecha fuera de la aplicación, y el respaldo ZIP reduce notablemente el tamaño gracias al relleno repetitivo.
- El manejo de errores evita que la aplicación se caiga ante archivos inexistentes, corruptos o sin permisos.